

Γραφική με Υπολογιστές 2026

Εργασία #2: Μετασχηματισμοί και Προβολές

Ζητούμενα

A. Συναρτήσεις Μετασχηματισμών

Να υλοποιηθεί η κλάση:

```
1 class Trafo:
2     """
3     Implements an affine transformation matrix
4     """
5     def __init__(self):
6         """
7         """
8         self.rot_mat = np.eye(3)
9         self.t_vec = np.zeros([3])
10
11    def translate(self, t_vec: np.ndarray) -> None:
12        """
13        Adds the specified translation vector to the object's
14        translation vector.
15        """
16
17    def rotate(self, axis: np.ndarray, angle: float, center: np.ndarray) -> None:
18        """
19        Calculates a rotation matrix from the specified axis angle components
20        and composes it the the object's rotation matrix
21        """
22
23    def xform_pnts(self, pts: np.ndarray) -> np.ndarray:
24        """
25        Transforms the incoming points w.r.t the object's
26        affine transformation
27        """
28
29    def sys2sys(self, sys_src: np.ndarray, pts: np.ndarray) -> np.ndarray:
30        """
31        Converts the incoming points defined in sys_src coordinate frame
32        to the coordinate frame specified by the object's rotation matrix
33        and an origin defined by t_vec.
34        """
```

Η οποία υλοποιεί τις λειτουργικότητες ενός affine μετασχηματισμού που περιλαμβάνει περιστροφή και μετατόπιση, και έχει ως μέλος έναν 4×4 πίνακα `mat`. Θα πρέπει να υλοποιηθούν οι συναρτήσεις μέλη:

- `translate(t_vec)`, η οποία προσθέτει το δοθέν διάνυσμα μετατόπισης στο διάνυσμα μετατόπισης του αντικειμένου `t_vec`.
- `rotate(axis, angle, center)`, η οποία υπολογίζει έναν πίνακα περιστροφής, που αντιστοιχεί σε δεξιόστροφη περιστροφή κατά γωνία `angle` σε rads, και περί άξονα με κατεύθυνση που δίνεται από το διάνυσμα `axis`, και τον συνθέτει στον πίνακα περιστροφής του αντικειμένου. `center` είναι το σημείο από το οποίο διέρχεται ο άξονας περιστροφής.
- `xform_pts(self, pts)` η οποία μετασχηματίζει τα σημεία `pts` βάσει του μετασχηματισμού που ορίζεται από το `rot_mat` και `t_vec` του αντικειμένου.
- `sys2sys(self, sys_src, pts)` η οποία μετασχηματίζει τα σημεία `pts` από το σύστημα συντ/νων `sys_src` $\in \mathbb{R}^{3 \times 3}$ στο σύστημα συντ/νων που προκύπτει από εφαρμογή περιστροφής `rot_mat` και μετατόπιση του σημείου αναφοράς `t_vec` της κλάσης.

B. Συνάρτηση προσανατολισμού κάμερας

Να υλοποιηθεί η συνάρτηση:

```

36 def lookout(eye: np.ndarray, up: np.ndarray, target: np.ndarray) -> Tuple[np.ndarray,
37     np.ndarray]:
38     # Calculate the camera's view matrix (i.e., its coordinate frame transformation
39     # specified
40     # by a rotation matrix R, and a translation vector t).
41     # :return a tuple containing the rotation matrix R (3 x 3) and a translation
42     # vector
43     # t (1 x 3)

```

Η συνάρτηση θα επιστρέφει τις παραμέτρους που χρειάζονται για το μετασχηματισμό σημείων από το WCS στο σύστημα συντ/νων της κάμερας. Αυτές θα είναι ο πίνακας περιστροφής $R \in \mathbb{R}^{3 \times 3}$, και το διάνυσμα μετατόπισης $t \in \mathbb{R}^3$. Πιο συγκεκριμένα:

- $eye \in \mathbb{R}^3$ είναι το σημείο του κέντρου της κάμερας.
- $up \in \mathbb{R}^3$ είναι το μοναδιαίο up διάνυσμα της κάμερας.
- $target \in \mathbb{R}^3$ είναι το σημείο στόχος.

Γ. Συνάρτηση προοπτικής προβολής με pinhole κάμερα

Να υλοποιηθεί η συνάρτηση:

```

41 def perspective_project(pts: np.ndarray, focal: float, R: np.ndarray, t: np.ndarray)
42     -> Tuple[np.ndarray, np.ndarray]:
43     # Project the specified 3d points pts on the image plane, according to a pinhole
44     # perspective projection model.

```

η οποία παράγει τις προοπτικές προβολές και το βάθος των σημείων εισόδου `pts`. Πιο συγκεκριμένα:

- $pts \in \mathbb{R}^{3 \times N}$ είναι ο πίνακας των σημείων εισόδου.

- $focal$ είναι η απόσταση του πετάσματος της κάμερας από το κέντρο (μετρημένη στις μονάδες που χρησιμοποιεί το σύστημα συντ/νων της κάμερας).
- $R \in \mathbb{R}^{3 \times 3}$ είναι ο πίνακας περιστροφής προς το σύστημα συντ/νων της κάμερας.
- $t \in \mathbb{R}^3$ είναι το διάνυσμα μετατόπισης προς το σύστημα συντ/νων της κάμερας.

Η συνάρτηση θα επιστρέφει τις 2D συντ/νες των σημείων εισόδου στο πέτασμα της κάμερας.

Δ. Συνάρτηση απεικόνισης

Να υλοποιήσετε τη συνάρτηση:

```
43 def rasterize(pts_2d: np.ndarray, plane_w: int, plane_h: int, res_w: int, res_h: int)
    -> np.ndarray:
44     # Rasterize the incoming 2d points from the camera plane to image pixel
    coordinates
```

η οποία απεικονίζει τις συντ/νες των σημείων από το σύστημα συντ/νων του πετάσματος της κάμερας, με πέτασμα $plane_h \times plane_w$, σε ακέραιες θέσεις (pixels) της εικόνας διάστασης $res_h \times res_w$.

Σημείωση: Ο άξονας της κάμερας περνά από το κέντρο ορθγώνιου διάστασης $plane_h \times plane_w$, ενώ η αρίθμηση του $res_h \times res_w$ πίνακα της εικόνας ξεκινά από τα κάτω προς τα πάνω και από αριστερά προς δεξιά και έχει τιμές $[0, \dots, res_w]$ οριζοντίως και $[0, \dots, res_h]$ κατακορύφως.

Ε. Συνάρτηση φωτογράφισης

Να υλοποιήσετε τη συνάρτηση:

```
45 def render_object(v_pos, v_clr, t_pos_idx, plane_h, plane_w, res_h, res_w, focal, eye
    , up, target) -> np.ndarray:
46     # render the specified object from the specified camera.
```

η οποία φωτογραφίζει μία 3D σκηνή ενός αντικειμένου από μία κάμερα. Πιο συγκεκριμένα:

- $v_pos \in \mathbb{R}^{N \times 3}$ είναι οι τρισδιάστατες συντ/νες των σημείων του αντικειμένου.
- $v_clr \in \mathbb{R}^{N \times 3}$ είναι ο πίνακας με τα χρώματα των κορυφών.
- t_pos_idx είναι ο πίνακας που περιέχει δείκτες σε σημεία του πίνακα v_pos που αποτελούν τις κορυφές των τριγώνων. Ο πίνακας είναι διάστασης $F \times 3$. Η i -οστή γραμμή του πίνακα, δηλώνει τις τρεις κορυφές που σχηματίζουν το τρίγωνο (με αναφορά σε κορυφές του πίνακα v_pos και η αρίθμησή του ξεκινάει από το 0).
- $plane_h$ και $plane_w$ είναι το ύψος και το πλάτος του πετάσματος της κάμερας.
- res_h και res_w είναι το ύψος και το πλάτος του καμβά σε pixel.
- $focal$ είναι η απόσταση του πετάσματος από το κέντρο της κάμερας.
- eye είναι το κέντρο της κάμερας ως προς το WCS.
- up είναι το up διάνυσμα της κάμερας.
- $target$ είναι το σημείο στόχος της κάμερας.

Ολά τα σημεία, και οι πίνακες που αφορούν 3D συντ/νες δίνονται σε μη ομογενή μορφή. Η συνάρτηση θα επιστρέφει έναν πίνακα $res_h \times res_w \times 3$ (τη φωτογραφία του αντικειμένου) και χρησιμοποιώντας κατάλληλα τις παραπάνω συναρτήσεις θα υλοποιεί όλο το pipeline της απεικόνισης ενός αντικειμένου. Επίσης θα πρέπει να χρησιμοποιεί τη συνάρτηση πλήρωσης της προηγούμενης εργασίας για να χρωματίσει το αντικείμενο με τη μέθοδο Gouraud.

Παραδοτέα

Έστω ότι το αντικείμενο προς φωτογράφιση βρίσκεται στην αρχή των αξόνων του κόσμου και ότι κάνει μία πλήρη περιστροφή γύρω από τον άξονα y σε διάρκεια $1sec$.

Τα demos που θα παραδώσετε θα πρέπει να προσομοιώνουν τα παρακάτω προβλήματα:

- Η κάμερα είναι στατική και φωτογραφίζει το αντικείμενο κατά τη διάρκεια της περιστροφής με ρυθμό $25fps$.
- Το αντικείμενο σταματάει την περιστροφή γύρω από τον άξονά του. Η κάμερα περιστρέφεται στην ακτίνα που σας δίνεται κοιτώντας πάντα το αντικείμενο και κάνει μία πλήρη δεξιόστροφη περιστροφή γύρω από τον άξονα y σε διάρκεια $1sec$, φωτογραφίζοντάς το με ρυθμό $25fps$.

Θα πρέπει να παραδώσετε:

1. Οι παραπάνω συναρτήσεις σε μορφή **σχολιασμένου** πηγαίου κώδικα python ($\geq v3.12$) με σχόλια γραμμένα στα **αγγλικά** ή **greeklish**, (κοινώς, **μη γράφετε σχόλια με ελληνικούς χαρακτήρες**).
2. Δύο demos που θα υλοποιούν τις δύο παραπάνω προσομοιώσεις. Οι προσομοιώσεις θα πρέπει να έχουν διάρκεια 1 δευτερόλεπτο, και να αποθηκεύονται 25 εικόνες ανά δευτερόλεπτο ($25fps$). Το αντικείμενο που σας δίνεται είναι η σφαίρα της προηγούμενης εργασίας. Θα χρησιμοποιήσετε τη συνάρτηση πλήρωσης με υφή από την προηγούμενη εργασία.

Σημείωση 1: Αν δεν είχατε υλοποιήσει τη συνάρτηση `render`, ή αν είχατε κάποιο λάθος στην υλοποίησή της, μπορείτε να χρησιμοποιήσετε τη `render` κάποιου/κάποιας συναδέλφου σας, **αρκεί να το δηλώσετε το όνομά του στην αναφορά σας**.

Σημείωση 2: Το αρχείο `hw2.npy` που σας δίνεται, περιέχει τις παραμέτρους του αντικειμένου `v_pos`, `v_uvs`, `t_pos_idx`, `diffuse.jpg` καθώς και όλες τις παραμέτρους που θεωρούνται γνωστές, όπως η ακτίνα του κύκλου κατά τον οποίο κινείται η κάμερα, και οι παράμετροι της κάμερας.

3. Αναφορά με:

- Περιγραφή της λειτουργίας και του τρόπου κλήσης των προγραμμάτων.
- Περιγραφή των συναρτήσεων.
- Τα ενδεικτικά αποτελέσματα που παράγονται από το demo.

Παρατηρήσεις

- Θεωρείστε ότι οι διαστάσεις της εικόνας είναι: `res_h = res_w = 512`.

- Οι εργασίες αξιολογούνται με χρήση Python(≥ 3.12).
- Οι εργασίες είναι **αυστηρά** ατομικές.
- Το background του καμβά είναι λευκό `rgb = np.array([1.0, 1.0, 1.0])`.
- Υποβάλετε ένα και μόνο αρχείο, τύπου .zip.
- Το όνομα του αρχείου πρέπει να είναι AEM.zip, όπου AEM είναι τα τέσσερα ψηφία του Α.Ε.Μ. του φοιτητή της ομάδας.
- Το προς υποβολή αρχείο πρέπει να περιέχει τα αρχεία κώδικα python και το αρχείο report.pdf το οποίο θα είναι η αναφορά της εργασίας.
- Η αναφορά πρέπει να είναι ένα αρχείο τύπου PDF, και να έχει όνομα report.pdf.
- Όλα τα αρχεία κώδικα πρέπει να είναι αρχεία κειμένου τύπου UTF-8, και να έχουν κατάληξη .py .
- Το αρχείο τύπου zip που θα υποβάλετε δεν πρέπει να περιέχει κανένα υπο-φάκελο.
- Για την ονομασία των αρχείων που περιέχονται στο προς υποβολή αρχείο, χρησιμοποιείτε μόνο αγγλικούς χαρακτήρες, και όχι ελληνικούς ή άλλα σύμβολα, πχ “#”, “\$”, “%” κλπ.

Προσοχή: Θα αξιολογηθούν μόνο όσες εργασίες έχουν demos που τρέχουν!